Course : AAD – ALGORITHM ANALYSIS AND DESIGN

Lecture 2

Asymptotic analysis

Dr Rahaf Ghazal

2025-2026

Asymptotic analysis

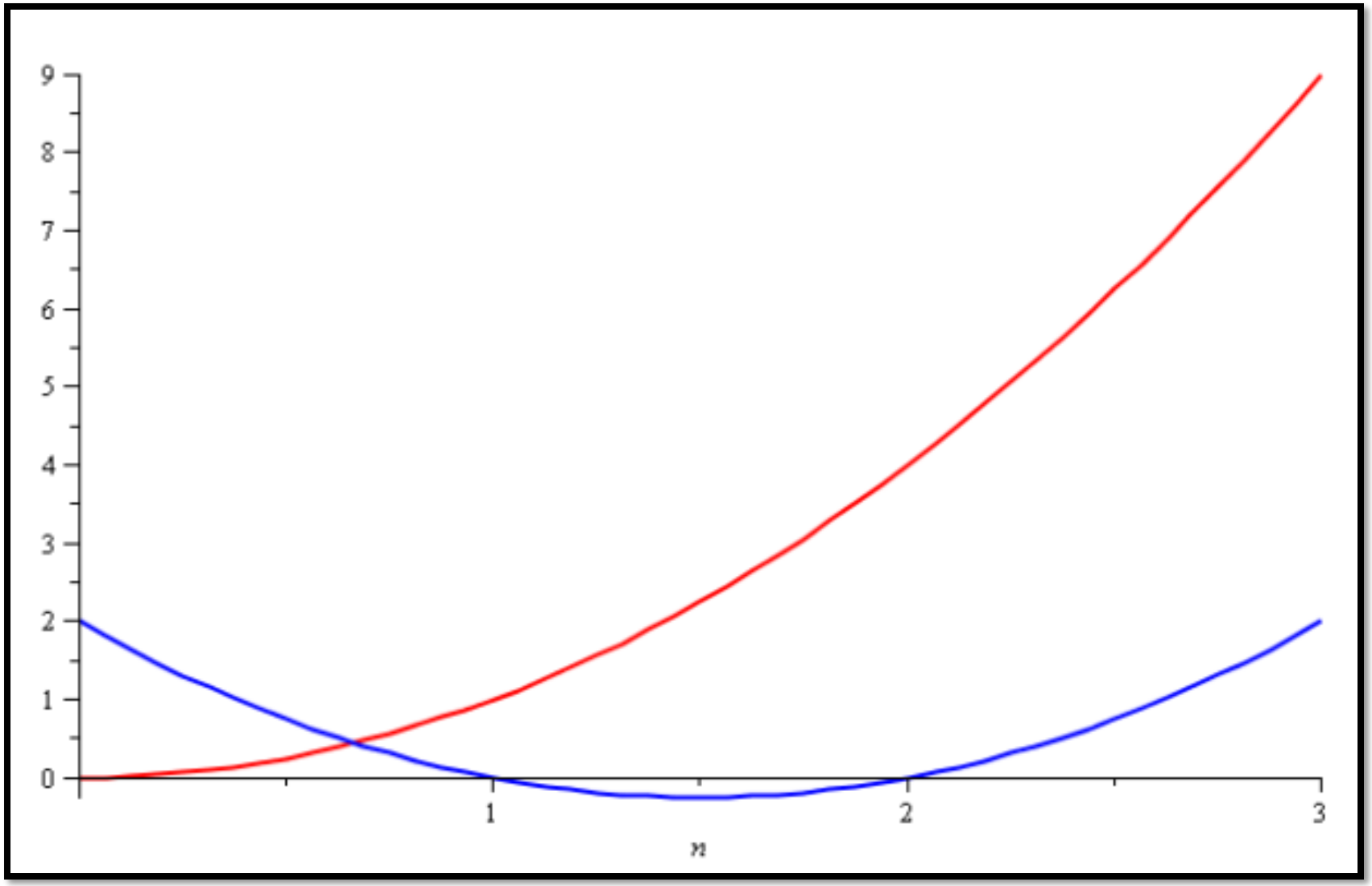
- It is a way to describe the characteristics of a function in the limit.
- It describes the rate of growth of functions.
- Focus on what's important by abstracting away low-order terms and constant factors.

Example

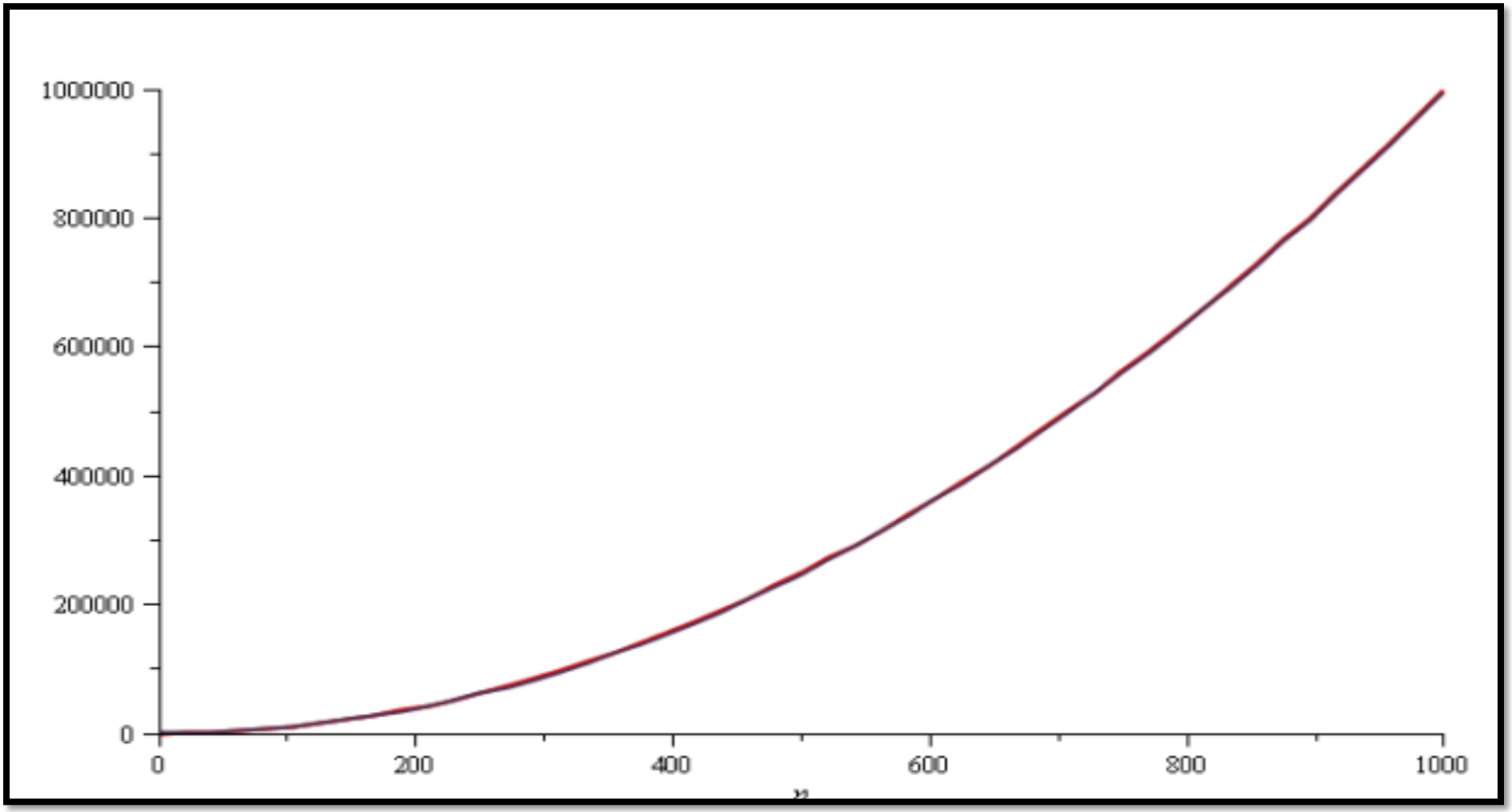
Consider the two functions

$f(n) = n^2$ and $g(n) = n^2 - 3n + 2$

Around $n = 0$, they look very different



But, For large values of n, we observe that the two functions converge very closely.



Asymptotic notation

- When we drop the constant coefficients and the less significant terms, we use asymptotic notation.
- Asymptotic notation is a mathematical shorthand used to describe the efficiency of an algorithm by focusing on its performance when input size grows very large.
- This is helpful when we want to **compare different algorithms and determine which will be faster** for significant inputs.
- The main types of asymptotic notation are:

Big O (O) , Big Omega (Ω), and **Big Theta (θ)**

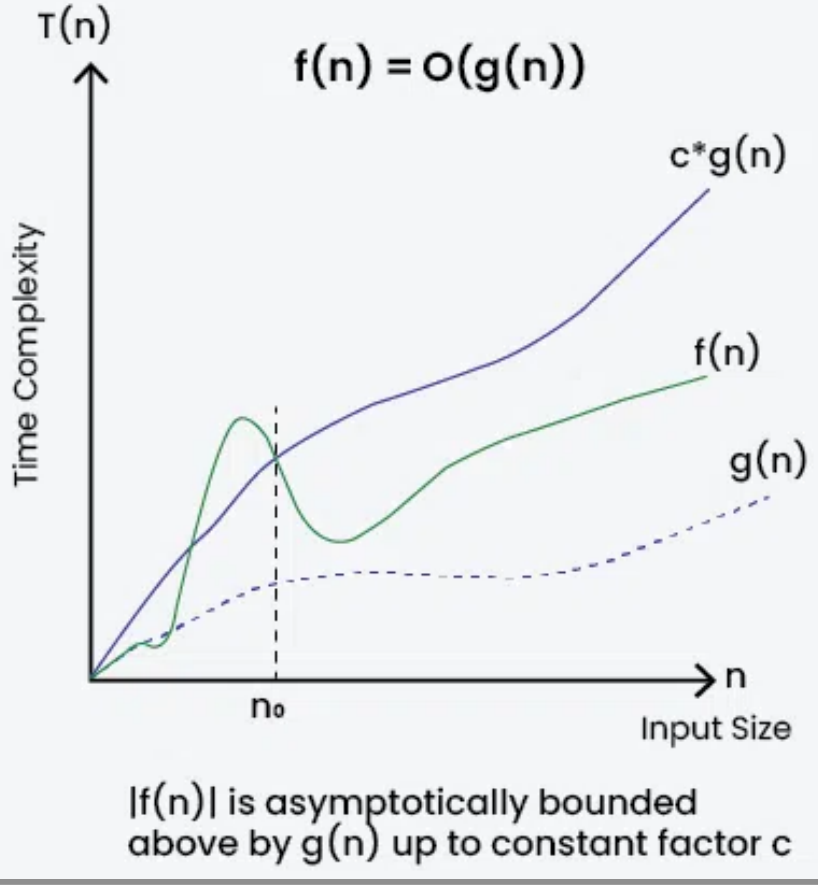
Big O Notation

Big-O is a way to express the upper bound of an algorithm's time. It analyses the worst-case situation of algorithm.

Big O Definition:

Given two functions **f(n)** and **g(n),** we say that **f(n)** is **O(g(n))** if **f(n)** grows no faster than **c*g(n)** for all $n \geq n_0$ where **c** and n_0 are constants.

Big O
Analysis

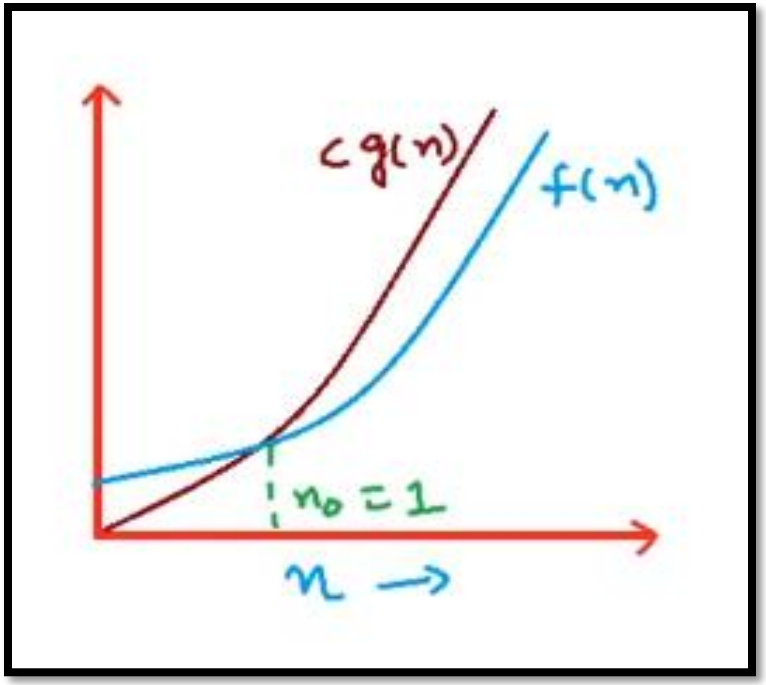


$f(n) = 5n^2 + 2n + 1 = O(n^2)$

$g(n) = n^2$

$c = 8, \quad f(n) \leq 8n^2, \quad n \geq 1$

$n_0 = 1$



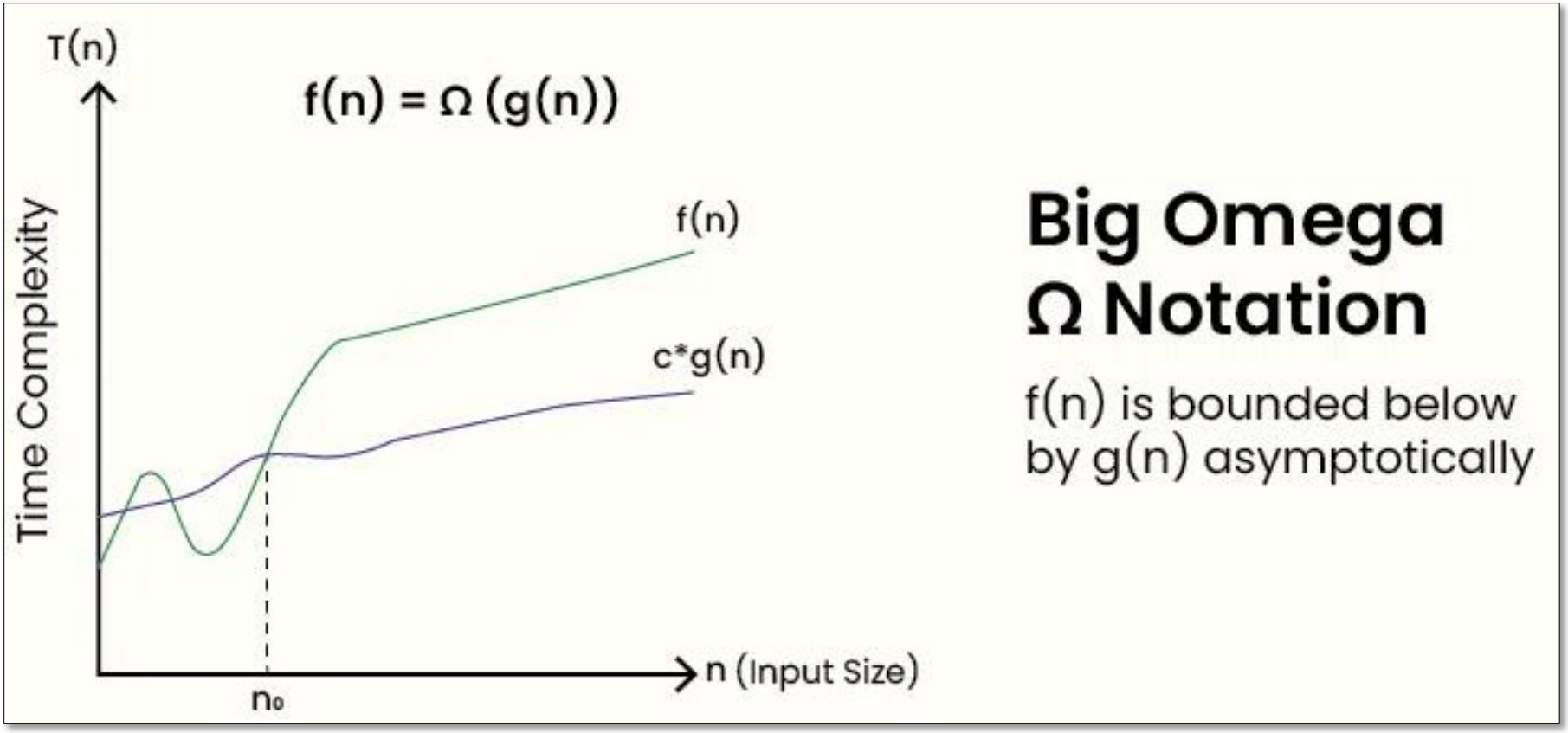
We usually consider one algorithm to be more efficient than another if its worst-case running time has a smaller order of growth

Big Ω Notation

Big-Ω is a way to express the lower bound of an algorithm's time. It analyses the best-case situation of algorithm.

Big Ω Definition:

Given two functions **f(n)** and **g(n),** we say that **f(n)** is **Ω(g(n))** if **f(n)** will always grow faster than **c*g(n)** for all $n \geq n_0$ where **c** and n_0 are constants.



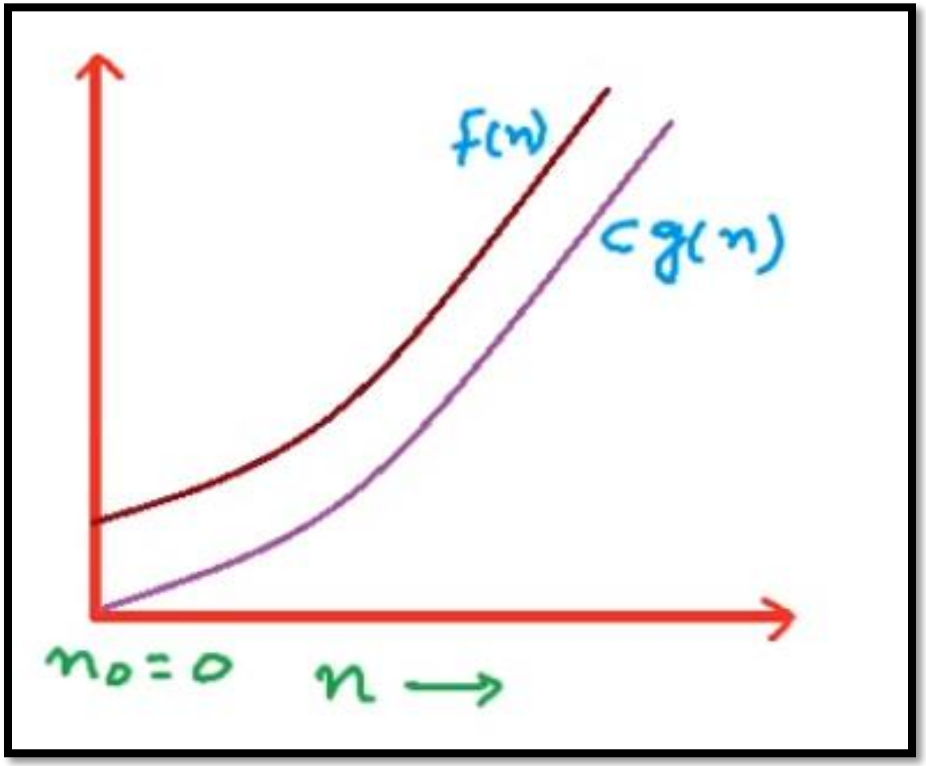
$f(n) = 5n^2 + 2n + 1 = \Omega(n^2)$

$g(n) = n^2$

$c = 5$

$n_0 = 0$

$5n^2 \leq f(n), \quad n \geq 0$

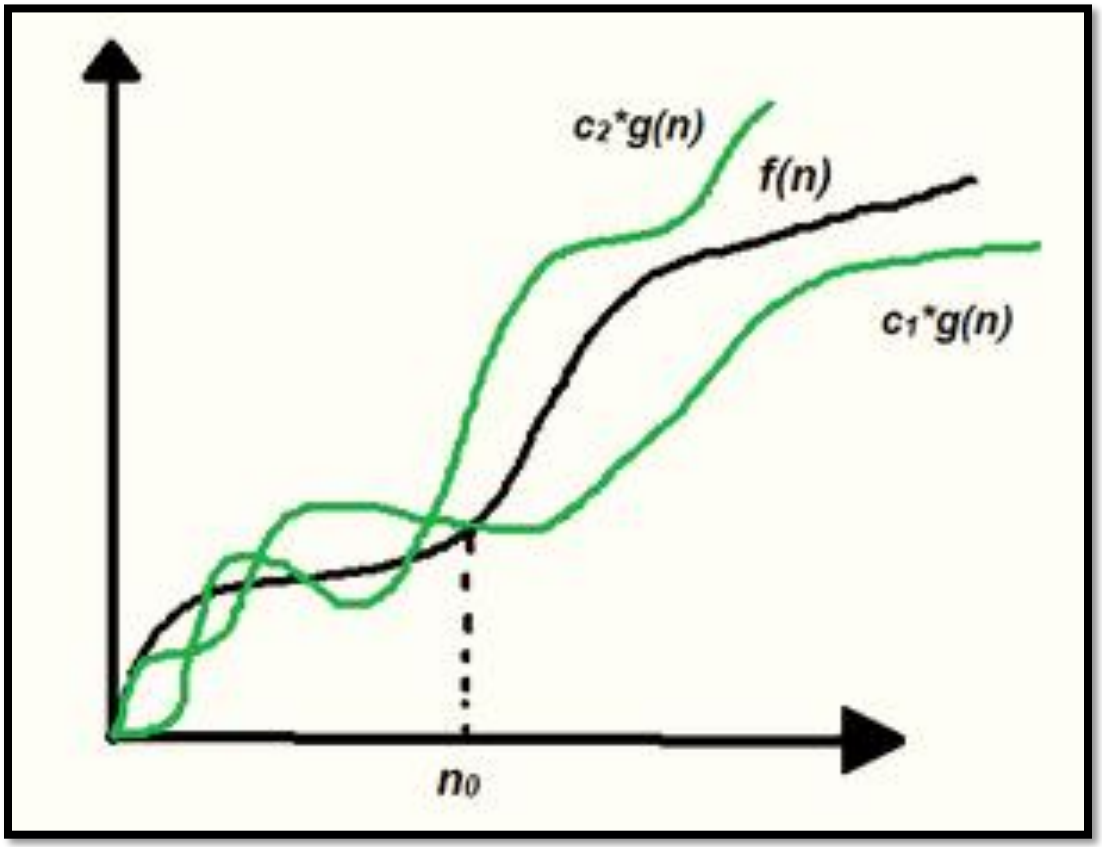


Big θ Notation

Big-θ is a way to express the average bound of an algorithm's time. It analyses the average-case situation of algorithm.

Big θ Definition:

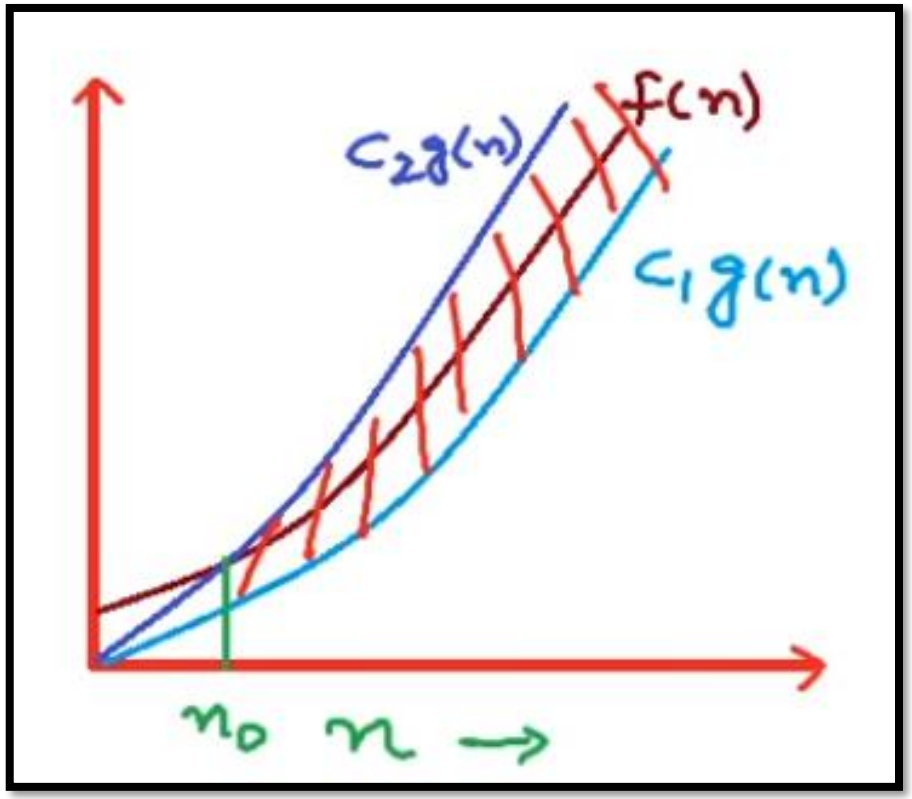
Given two functions **f(n)** and **g(n),** we say that **f(n)** is **θ(g(n))** if the value **f(n)** is always between
c1 * g(n) and c2 * g(n) for large values of **n (n ≥ n0)**



$f(n) = 5n^2 + 2n + 1 = \Theta(n^2)$

$g(n) = n^2$

$c_1 = 5,\ c_2 = 8,\ n_0 = 1$



Asymptotic Limit Theorems

If we want to determine the growth of a function, we just have to apply a limit at infinity for relationship between *f* and *g*, as following:

$$\text{If } \lim_{n\to\infty}\left(\frac{f(x)}{g(x)}\right) = 0, \text{ then } f(x) \text{ is } O(g(x))(\text{Big} - \text{O})$$

$$\text{If } \lim_{n\to\infty}\left(\frac{f(x)}{g(x)}\right) = \infty, \text{ then } f(x) \text{ is } \Omega(g(x)) \text{ (Big-Omega)}$$

$$\text{If } \lim_{n\to\infty}\left(\frac{f(x)}{g(x)}\right) = L, \text{ where } 0 < L < \infty, \text{ then } f(x) \text{ is } \Theta(g(x)) \text{ (Big-Theta)}$$

Let's look at a few examples.

Example 1

1. If $f(x) = 9 + 4x^2$ and $g(x) = x^2$, does

$$f(x) = O(g(x)), \Omega(g(x)), \text{ or } \Theta(g(x))?$$

$$\lim_{x \to \infty} \left(\frac{f(x)}{g(x)} \right) = \lim_{x \to \infty} \left(\frac{9 + 4x^2}{x^2} \right) = \frac{4}{1} = 4$$

Since 4 is a positive value between zero and infinity, f(x) is big-Theta of g(x). Using our limit theorems, we can say:

$$f(x) = \Theta\left(x^2\right)$$

Furthermore, we now know that there are constant values constants c_1, c_2, and k, such that f(x) is both $O\left(g\left(x\right)\right)$ and $\Omega\left(g\left(x\right)\right)$.

Example 2

2. If $f(x) = 3x^3 - 5x$ and $g(x) = x^2$, does

$$f(x) = O(g(x)), \Omega(g(x)), \text{ or } \Theta(g(x))?$$

$$\lim_{x \to \infty} \left(\frac{f(x)}{g(x)} \right) = \lim_{x \to \infty} \left(\frac{3x^3 - 5x}{x^2} \right) = \infty$$

This indicates that f(x) is big-Omega of g(x). Using our limit theorems, we can say:

$$f(x) = \Omega\left(x^2\right)$$

Example 3

3. If $f(x) = 7\log x$ *and* $g(x) = x$, *does*

$$f(x) = O(g(x)), \Omega(g(x)), \text{ or } \Theta(g(x))?$$

$$\lim_{x\to\infty}\left(\frac{f(x)}{g(x)}\right) = \lim_{x\to\infty}\left(\frac{7\log x}{x}\right) = \frac{\infty}{\infty} \overset{\text{Appply L. Hopital's Rule}}{=} \lim_{x\to\infty}\left(\frac{\left(\frac{7}{x}\right)}{1}\right) = \lim_{x\to\infty}\left(\frac{7}{x}\right) = \frac{7}{\infty} = 0$$

$$\lim_{x\to\infty}\left(\frac{7\log x}{x}\right) \overset{L\text{ 'Hopital's Rule}}{=} \lim_{x\to\infty}\left(\frac{\left(\frac{7}{x}\right)}{1}\right) = \lim_{x\to\infty}\left(\frac{7}{x}\right) = \frac{7}{\infty} = 0$$

This indicates that f(x) is big-Oh of g(x). Using our limit theorems, we can say:

$$f(x) = O(x)$$

مثال: حساب التعقيد الزمني الوسطي لمسألة البحث التسلسلي عن العنصر x ضمن قائمة تحوي n عنصراً .

- من الواضح أن أسوأ الحالات الممكنة هي وجود العنصر المطلوب في نهاية القائمة و بالتالي :

$$\mathbf{MaxT(n) = n = O(n)}$$

- بينما أحسن حالة هي وجود العنصر المطلوب في أول القائمة أي نقوم بعملية مقارنة واحدة فقط :

$$\mathbf{MinT(n) = 1 = \Omega(1)}$$

- أما لحساب التعقيد الزمني الوسطي نحدد ما يلي:

مع الأخذ بعين الاعتبار أن العنصر x يمكن أن يكون موجوداً في القائمة و يمكن لا. نفرض أن q هو احتمال وجود x في القائمة و لنفرض أنه إذا وجد x في القائمة فإن المواضع كلها متساوية الاحتمال.

فإذا كنا نعلم سلفاً أن x موجود في القائمة فإن q=1 و بالتالي :

$$\boldsymbol{Average_T(n) = \frac{n+1}{2} = \theta(n)}$$

و إذا كان احتمال وجود x في القائمة q=1/2 فإن:

$$\boldsymbol{Average_T(n) = \frac{3n+1}{4} = \theta(n)}$$

To compare between algorithms, we usually focus on studying the worst-case scenario, Big-O(T(n))

A Quick Way to find Big O of an Expression

- Ignore the lower order terms and consider only highest order term.
- Ignore the constant associated with the highest order term.

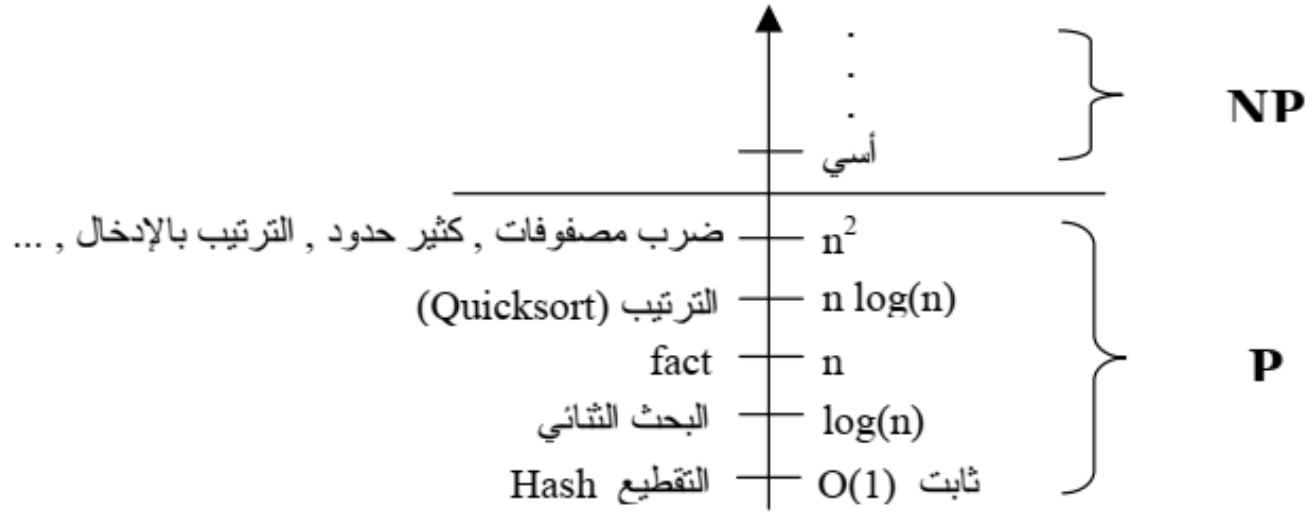
Example 1: $f(n) = 3n^2 + 2n + 1000Logn + 5000$
After ignoring lower order terms, we get the highest order term as $3n^2$
After ignoring the constant 3, we get n^2
Therefore the Big O value of this expression is $O(n^2)$

Example 2 : $f(n) = 3n^3 + 2n^2 + 5n + 1$
Dominant Term: $3n^3$
Order of Growth: Cubic (n^3)
Big O Notation: $O(n^3)$

(Polynomial) P and (Non Polynomial) NP problems

- ✓ نقول عن خوارزمية أنها **فعالة** (P = polynomial) إذا كانت درجة تعقيدها ذات درجة كثير حدود .
- ✓ نقول عن خوارزمية أنها **غير فعالة** (NP = non polynomial) إذا كانت درجة تعقيدها ليست بدرجة كثير حدود أي أسية.
- ✓ إن المسائل التي تنتمي إلى P هي مسائل سهلة و قابلة للمعالجة بينما المسائل التي تنتمي إلى NP هي مسائل صعبة و غير قابلة للمعالجة إلا في حالة معطيات ذات حجم محدود .

يبين الشكل التالي أمثلة على درجات تعقيد مختلفة مع انتمائهم إلى P أو NP :



The forms of complexity functions we encounter in most problems (with respect to the size of the data)

$O(1)$	constant (same as $O(k)$ for constant k)
$O(\log n)$	logarithmic
$O(n)$	linear
$O(n \log n)$	"n log n"
$O(n^2)$	quadratic
$O(n^3)$	cubic
$O(n^k)$	polynomial (where is k is any constant)
$O(k^n)$	exponential (where k is any constant > 1)

- للمقارنة نعتمد على الحقائق التالية :

$$O(1) \le O(\log n) \le O(n^k) \le O(n) \le O(n \log n) \le O(n^c) \le O(c^n) \le O(n!) \quad ; (0 \le k \le 1 \le c)$$

Examples of Big-O of some codes

- التعقيد الزمني الثابت من الشكل O(1) مثل:

```
x:=y;    if(x<12)    z:=6*n+9;
```

- التعقيد الزمني المتغير للحلقات (التي يتغير عدادها عن طريق معاملي الجمع أو الطرح)

من الشكل O(n) مثل:

```
for (int j = 1; j <=n; j += c) {
   // some O(1) expressions
}
```

- التعقيد الزمني للحلقات المتداخلة (التي يتغير عدادها عن طريق معاملي الجمع أو الطرح) من الشكل $O(n^2)$ مثل:

```
for (int i = 1; i <=n; i += c) {
    for (int j = 1; j <=n; j += c) {
      // some O(1) expressions
    }
}
```

Examples of Big-O of some codes

- التعقيد الزمني للحلقات (التي يتغير عدادها عن طريق معاملي الضرب أو القسمة) من الشكل $O(Log_c n)$ مثل:

```
for (int i = 1; i <=n; i *= c) {
    // some O(1) expressions
}
```

- التعقيد الزمني للحلقات (التي يتغير عدادها بشكل أسي) من الشكل $O(Log_c Log_2(n))$ مثل:

```
// Here c is a constant greater than 1
for (int i = 2; i <=n; i = pow(i, c)) {
    // some O(1) expressions
}
```

Exercise 1: analysis the following algorithm and Give an idea of its complexity:

```
void Main()
                {
                int m=1, p=3, l=5;
                for (int i=1; i<=n; i++)
                        {
                                m=m+1;
                                p=p*2;
                                l=l+3;
                        }
                }
```

$$\sum_{i=1}^{n} 3 * 1 = 3 * \sum_{i=1}^{n} 1 = 3(n-1+1) = 3n = O(n)$$

Exercise 2: analysis the following algorithm and Give an idea of its complexity:

```
void Main()
                {
                int m=1, p=3, l=5;
                for (int i=1; i<=n; i++)
                   for (int j=1; j<=n; j++)
                       {
                              m=m+1;
                              p=p*2;
                              l=l+3;
                       }
```

$$\sum_{i=1}^{n}\sum_{j=1}^{n} 3 = 3 * \sum_{i=1}^{n}\sum_{j=1}^{n} 1 =$$

$$3\sum_{i=1}^{n} (n-1+1) = 3\sum_{i=1}^{n} n =$$

$$3 * n\sum_{i=1}^{n} 1 = 3 * n * n = 3\,n^2 = O(n^2)$$

Exercise 3: analysis the following algorithm and Give an idea of its complexity:

```
void Main()
                {
                int m=1, p=3, l=5;
                for (int i=1; i<=n; i++)
                   for (int j=1; j<=i; i++)
                        {
                                m=m+1;
                                p=p*2;
                                l=l+3;
                        }
                }
```

$$\sum_{i=1}^{n}\sum_{j=1}^{n} 3 = 3 * \sum_{i=1}^{n}\sum_{j=1}^{i} 1 =$$

$$3\sum_{i=1}^{n} (i-1+1) = 3\sum_{i=1}^{n} i = 3\frac{n(n+1)}{2}$$

$$= O(n^2)$$

مثال على التعقيد الزمني لخوارزمية حساب العاملي:

- العملية الرئيسية هى عملية الضرب

$$
\begin{aligned}
T(0) &= 0 \\
T(n) &= 1 + T(n-1) \\
&= 2 + T(n-2) \\
&= 3 + T(n-3) \\
&\cdots\cdots\cdots\cdots\cdots\cdots \\
&= k + T(n-k) \quad ; \quad n - k = 0 \quad \Rightarrow \quad k = n \quad \Rightarrow \\
&= n + T(0) \\
&= n + 0 \\
T(n) &= O(n)
\end{aligned}
$$

ملاحظة: دراسة تحليل الخوارزميات العودية لها طرق مختلفة سنتطرق إليها في محاضرات قادمة.

لذا سنكتفي بعرض المثال السابق عن تحليل التوابع العودية في هذا القسم.

Homework: analysis the following algorithms and Give an idea of its complexity:

```
public void function(int n) {
        for(int i = 1 ; i <= n/3 ; i + + )
            for(int j = 1 ; j <= n ; j += 4 )
                System.out.println(" * ");
}
```

```
algorithm add(a,b,b,m,n)
{for i:=1 to m do
for j:=1 to n do
c[i,j]:=a[i,j]+b[i,j]; }
```

```
public void Function(int n) {
        int i=1, s=1;
        while( s <= n) {
            i++;
            s= s+i;
            System.out.println("*");
        }
}
```

```
Algorithm sum(a,n)
{ s=0.0;
for i: =1 to n do
{ s:=s+a[i];  }
return s;      }
```